

Memo Formation — Onboarding Commercetools

LE PROJET EN 30 SECONDES

Migration Salesforce SFRA → headless moderne **3 marques, 1 codebase** : Etam · Undiz · Maison 123

```
CommerceTools  →  ASTRO (SSR)  ←  Contentstack
(e-commerce)    + Preact 3KB   (CMS headless)
                + Tailwind
                ↑
                BFF Node.js
                (auth/customer)
```

ISLANDS ARCHITECTURE

HTML statique par défaut, JS uniquement là où c'est nécessaire.

```
Page HTML statique
|
+-- [Island] ProductGallery  client:visible → JS au scroll
|
+-- [Island] AddToCartButton client:load    → JS immédiat
|
Page HTML statique
```

Directives :

- **client:load** → JS immédiat (cart, boutons critiques)
- **client:visible** → JS au scroll (galerie, carrousel)
- **client:idle** → JS quand idle (newsletter, analytics)
- **client:only** → Client-only, pas de SSR (auth, devtools)

FLUX DE DONNÉES

Flux A — API Routes (panier, wishlist)

```
Browser
-> fetch("/api/cart/add")
-> cart.service.ts (Service Layer)
-> commercetools.ts (admin token)
-> CT API
```

Flux B — BFF direct (auth, customer)

Browser

- > Signal action (auth.store.ts)
- > lib/api/auth.ts (apiFetch)
- > BFF (PUBLIC_BFF_BASE_URL)
- > CT API

Actions Signal Store -> Routes BFF :

```
login()           -> signIn()           POST /auth/signin
register()        -> signUp()          PUT  /auth/signup
updateProfile()  -> updateCustomer()  POST /customer/me
changePassword() -> changePassword()  PATCH /customer/me/password
```

Routes BFF — Reference

ROUTE	METHODE	FONCTION JS	AUTH?
/auth/signin	POST	signIn(email, pw)	Non
/auth/signup	PUT	signUp({email, pw, ...})	Non
/auth/guest	POST	guestToken()	Non
/auth/refresh	POST	refreshToken(token)	Non
/auth/isEmailExists	GET	checkEmailExists(email)	Non
/customer/me	GET	fetchMe(token)	Bearer
/customer/me	POST	updateCustomer(token, d)	Bearer
/customer/me/password	PATCH	changePassword(t, old, n)	Bearer

Migration : Flux A migrera vers Flux B. Objectif : tout passe par le BFF.

QUAND UTILISER QUOI ?

BESOIN	PATTERN	EXEMPLE
SEO / First Paint	SSR frontmatter	Fiche produit, listing
Panier / wishlist	Signal + API Route	Add to cart
Auth / Customer	Signal + BFF	signIn, fetchMe...
Non-critique	useEffect	Cross-sell, recos
Etat partage	Signal store	Cart, session

COUCHES D'ABSTRACTION

Flux A — Cote serveur (Astro)

```

Page Astro / API Route
|
v
SERVICE LAYER      lib/services/*.service.ts
|
v
DOMAIN LAYER        lib/domain/*.ts
|
v
API LAYER            lib/api/commercetools.ts
                     SERVER-ONLY, admin token

```

Flux B — Cote client (BFF)

```

Composant Preact (action)
|
v
SIGNAL STORE          lib/signals/auth.store.ts
                     State + Computed + Actions
|
v
API MODULES           lib/api/auth.ts
                     lib/api/customer.ts
|
v
HTTP CLIENT           lib/api/http-client.ts
                     apiFetch() -> PUBLIC_BFF_BASE_URL

```

SIGNAL vs useState

Signal = etat PARTAGE

- Partage entre composants non lies
- Persiste entre les pages
- Pas de prop drilling
- Ex : cart, user, filters, wishlist

useState = etat LOCAL

- Isole a un seul composant
- Ephemere (disparait au demontage)
- Ex : selectedSize, isLoading, errorMessage

Les deux ensemble :

```
import { addToCart } from "../../signals/cart";
```

```
export function AddToCartButton({ product }) {
  const [isLoading, setIsLoading] = useState(false);

  const handleClick = async () => {
    setIsLoading(true);
    await addToCart(product);
    setIsLoading(false);
  };

  return (
    <button onClick={handleClick}>
      {isLoading ? "..." : "Ajouter"}
    </button>
  );
}
```

Anti-pattern : prop drilling

```
// NON <ProductCard onAddToCart={addToCart} />
// OUI import { addToCart } from "../signals/cart";
```

NAMING CONVENTIONS

Signals → noms descriptifs

```
export const cart = signal(null); // state
export const cartCount = computed(...); // computed
export async function addToCart() { } // action
```

useState → noms prefixes

```
OK const [selectedSize, setSelectedSize] = useState(null);
OK const [isLoading, setIsLoading] = useState(false);
NON const [size, setSize] = useState(null);
NON const [loading, setLoading] = useState(false);
```

SIGNALS : OU LES UTILISER ?

AUTORISE	INTERDIT
signals/*.ts	pages/*.astro (frontmatter)
components/**/*.tsx	lib/services/*.ts
lib/api/auth.ts (BFF)	lib/api/commercetools.ts
lib/api/customer.ts (BFF)	

Signals = cote client uniquement (browser). auth.ts / customer.ts = modules CLIENT (BFF)
commercetools.ts = SERVER-ONLY (admin token)

SIGNALS DEVTOOLS (Chrome)

Extension Chrome pour visualiser les signals en temps reel.

```
1. lib/signals/index.ts    <- barrel export
2. SignalsDevtools.tsx    <- init provider (DEV)
3. Layout.astro           <- client:only="preact"
```

Nouveau signal store ? Ajoute-le dans index.ts avec alias : loading as cartLoading

COMMERCETOOLS — L'ESSENTIEL

- 1 projet, 3 stores (Etam, Undiz, M123) chaque store a son catalogue et ses prix
- Panier : cartId en cookie anonyme -> client apres login
- Versioning : chaque update necessite la version
- 2 types de token :

```
Admin token (serveur, OAuth2 client_credentials)
  createCartInStore()
  fetchCartByAnonymousId()

Customer token (gere par le BFF)
  signIn()          -> POST /auth/signin
  fetchMe(token)    -> GET  /customer/me
  updateCustomer() -> POST /customer/me
  changePassword() -> PATCH /customer/me/password
```

CONTENTSTACK — CMS

- Headless : Content Delivery API -> Astro SSR -> HTML
- Live Preview : modifs en temps reel (Visual Builder)
- Content Types : Pages, Composants, Globaux
- CLI : csdx pour auth, export/import

Live Preview :

```
isPreview=false (prod)
-> Donnees SSR pre-fetchees, pas de JS preview
```

```
isPreview=true (Visual Builder)
  -> SDK client + addEditableTags() -> data-cslp
```

- `addEditableTags()` -> propriete \$ sur chaque entry
- `getEditableProps(obj, "field")` -> extrait data-cslp
- `{...editable?.titleProps}` -> applique sur le HTML

Guide complet : </guides/live-preview>

METHODOLOGIE — CHECKLIST FEATURE

- | | |
|----------------|--|
| 1. Analyser | Specs, maquettes, contexte |
| 2. Identifier | Quelles couches ? |
| 3. Implementer | Bottom-up : API -> Domain -> Service -> Signal -> UI |
| 4. Integrer | Directive : load / visible / idle ? |
| 5. Tester | Edge cases, responsive |
| 6. Review | Naming, conventions, PR |

STRUCTURE DU PROJET

```
src/
  pages/           Routes Astro + API Routes
  features/        Features metier (cart/, product/)
  components/      UI reutilisables
  lib/
    api/
      commercetools.ts  SERVER-ONLY (admin token)
      contentstack.ts   SERVER-ONLY
      http-client.ts    CLIENT -> BFF (apiFetch)
      auth.ts           CLIENT (signIn, signUp...)
      customer.ts       CLIENT (fetchMe, updateCustomer...)
    domain/            Transformation CT -> UI
    services/          Orchestration (API + Domain)
    signals/            Signal stores (client-side)
  layouts/            Layouts Astro
  styles/             CSS global, tokens
```

LIENS RAPIDES

```
Architecture
  /architecture/overview
  /architecture/islands
  /architecture/data-flow
```

Stack

- /stack/astro
- /stack/preact
- /stack/commercetools
- /stack/contentstack
- /stack/tailwind

Guides

- /guides/methodology
- /guides/data-fetching
- /guides/live-preview
- /guides/contentstack-cli

Exemples

- /examples/components
- /examples/api-routes
- /examples/signals